

QuizByte

A containerized two-tier MCQ quiz application

Student	Shashank
Roll No.	231109
Course	CS515 — Unix Programming
Deadline	29 March 2026
GitHub	https://github.com/shashank-6832/quiz-app
Stack	React · Node.js/Express · PostgreSQL · Docker · Kubernetes

1 — Project Overview

QuizByte is a full-stack, two-tier web application that lets users take a randomized 10-question multiple-choice quiz covering Computer Science topics including algorithms, networking, DevOps, data structures, and operating systems.

After completing a quiz, the player's score is saved to a PostgreSQL database and a live leaderboard displays the top 10 scores across all sessions.

Architecture

Layer	Technology	Role
Frontend	React 18 + Vite + Nginx	User interface — quiz flow, score display
Backend	Node.js + Express	REST API — serves questions, saves scores
Database	PostgreSQL 15	Persistent storage for questions and scores
Container	Docker + Docker Compose	Local multi-container orchestration
Cluster	Kubernetes (Minikube)	Production-grade local deployment

2 — Phase 1: Application Development & Version Control (Git)

The repository was initialized locally and pushed to GitHub. Commits were made incrementally to reflect progressive development across all three phases.

Git Setup Commands

```
git init
git add .gitignore README.md
git commit -m "Initial project structure and README"
git add backend/package.json backend/index.js
git commit -m "Add Express backend with health route"
git add backend/Dockerfile
git commit -m "Add multi-stage Dockerfile for backend"
git add docker-compose.yml
git commit -m "Add docker-compose.yml for local multi-container setup"
git add frontend/
git commit -m "Add React frontend with Vite, components, and Nginx config"
git add k8s/
git commit -m "Add Kubernetes manifests for postgres, backend, and frontend"
git remote add origin https://github.com/shashank-6832/quiz-app.git
git branch -M main
git push -u origin main
```

Screenshot — git log --oneline

```
179a72f (HEAD → main, origin/main) Add Kubernetes manifests for postgres, backend, and frontend
d85496b Add React frontend with Vite, components, and Nginx config
bc35fe2 Add docker-compose.yml for local multi-container setup
e2dd801 Add multi-stage Dockerfile for backend
538d4bc Add Express backend with health route
96f98bd Initial project structure and README
(END)
```

Terminal output of git log --oneline showing 6 incremental commits

2 — Phase 2: Containerization (Docker)

Both the frontend and backend use multi-stage Dockerfiles. The frontend's production image contains only Nginx and static files — no Node.js at runtime. Docker Compose orchestrates all three services locally with health checks and automatic retry logic.

Build & Run Commands

```
# Build images individually
docker build -t quiz-backend ./backend
docker build -t quiz-frontend ./frontend

# OR build and run all services together
docker compose up --build

# App available at http://localhost:3000

# Useful verification commands
docker ps
docker images
docker compose logs -f backend
docker compose down
```

Key Docker Design Decisions

Multi-stage builds	Frontend image contains only Nginx + static files. No Node.js in production.
Non-root user	Backend container runs as a dedicated non-root user (quizuser) for security.
Health checks	PostgreSQL has a health check; Docker Compose waits before starting the backend.
DB retry logic	Backend retries the PostgreSQL connection up to 10 times on startup.
Nginx reverse proxy	Frontend Nginx proxies /api/* to the backend container internally.

3 — Phase 3: Orchestration (Kubernetes / Minikube)

The application is deployed to a local Minikube cluster using three manifest files. Both backend and frontend run with 2 replicas. PostgreSQL data is persisted via a PersistentVolumeClaim. The frontend is exposed via NodePort on port 30080.

Manifest Files

k8s/postgres.yaml	Secret (credentials), PersistentVolumeClaim, Deployment (1 replica), ClusterIP Service
k8s/backend.yaml	ConfigMap (env vars), Deployment (2 replicas), ClusterIP Service
k8s/frontend.yaml	Deployment (2 replicas), NodePort Service (port 30080)

Step-by-Step Kubernetes Deployment Commands

```
# 1. Start Minikube cluster
minikube start

# 2. Point Docker CLI to Minikube's internal Docker daemon
eval $(minikube docker-env)

# 3. Build images inside Minikube's Docker context
docker build -t quiz-backend:latest ./backend
docker build -t quiz-frontend:latest ./frontend

# 4. Apply manifests – order matters (DB first)
kubectl apply -f k8s/postgres.yaml
kubectl apply -f k8s/backend.yaml
kubectl apply -f k8s/frontend.yaml

# 5. Verify all pods are Running
kubectl get pods

# 6. Open app in browser via Minikube tunnel
minikube service quiz-frontend

# OR: get Minikube IP and visit http://<ip>:30080
minikube ip
```

Screenshot — kubectl get pods

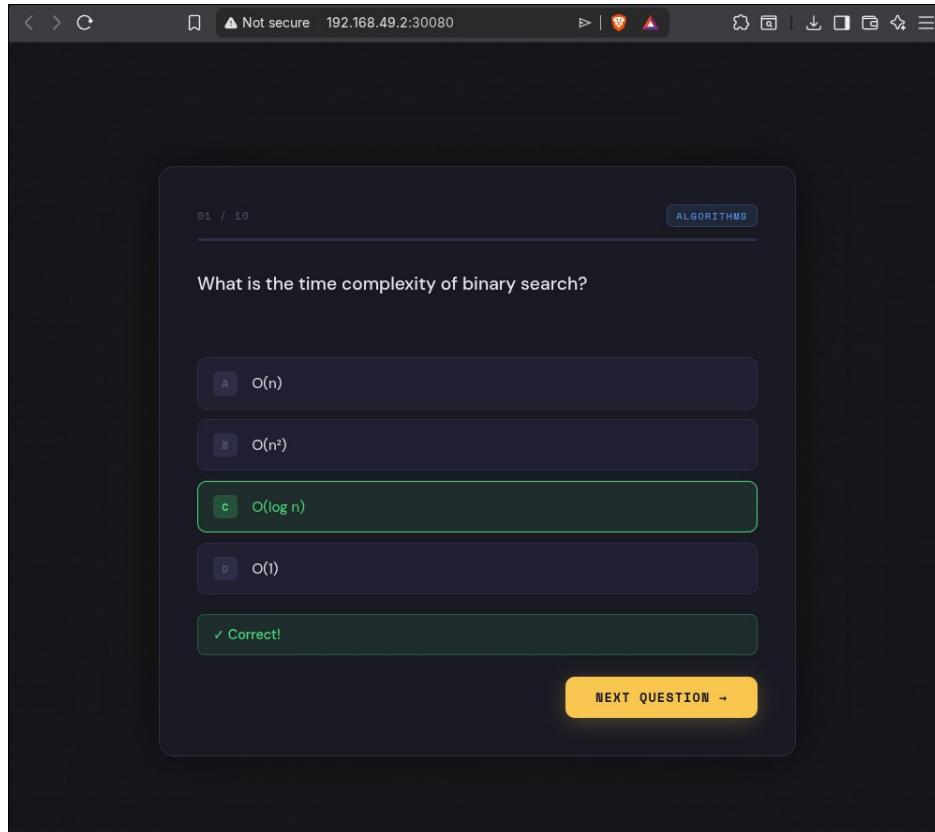
```
shashank@fedora:assignment3/quiz-app <main*>$ kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
postgres-5df6bc78dd-8tr7t           1/1     Running   0           15m
quiz-backend-7f8f54b974-5r8f9       1/1     Running   0           15m
quiz-backend-7f8f54b974-x4jnk       1/1     Running   0           15m
quiz-frontend-5d5957769c-b5s58      1/1     Running   0           15m
quiz-frontend-5d5957769c-nlxpc      1/1     Running   0           15m
shashank@fedora:assignment3/quiz-app <main*>$
```

kubectl get pods — all 5 pods Running: 1 postgres, 2 backend replicas, 2 frontend replicas

4 — Proof of Execution: Application Running in Browser

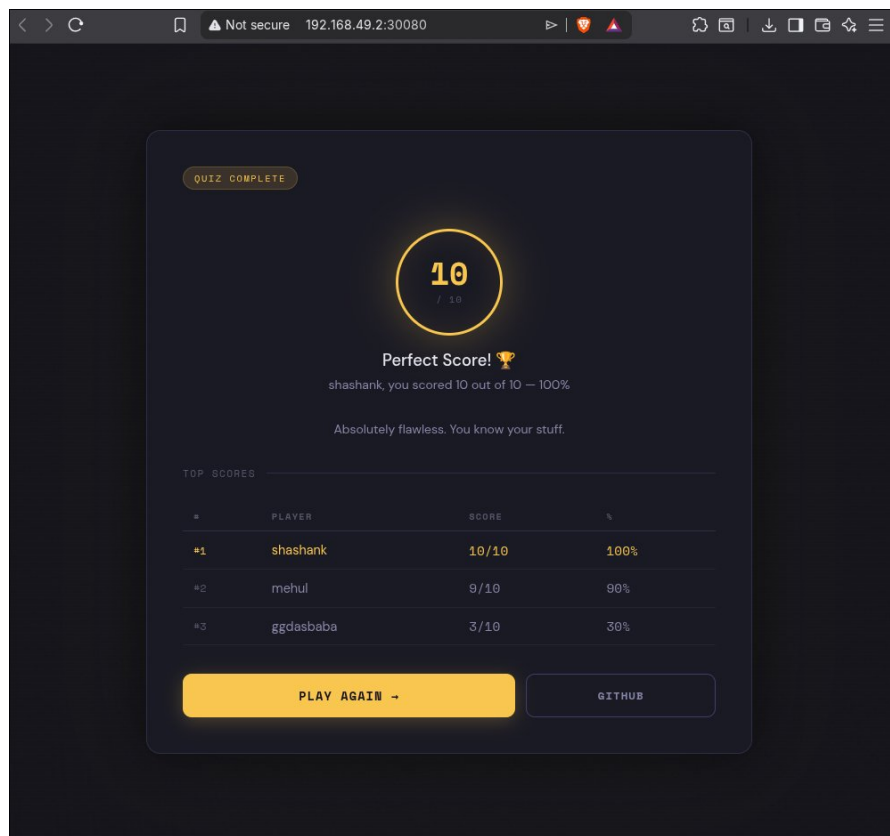
The following screenshots show QuizByte successfully running in the browser after being served through the Kubernetes NodePort service via Minikube (192.168.49.2:30080). The first screenshot shows an active quiz question with the correct answer highlighted. The second shows the result screen with a perfect score of 10/10 and the live leaderboard.

Quiz Screen — Correct Answer Highlighted



QuizByte running at 192.168.49.2:30080 — Question 1/10, correct answer selected ($O(\log n)$)

Result Screen — Score and Leaderboard



Result screen showing perfect score 10/10 (100%) with live leaderboard from PostgreSQL

5 — API Reference

Method	Endpoint	Description
GET	/api/health	Health check — returns server status and timestamp
GET	/api/questions	Returns all quiz questions in randomized order
POST	/api/scores	Save a player score { player_name, score, total }
GET	/api/scores	Get top 10 scores ordered by score descending

6 — Errors Encountered & Fixes

Unable to add remote 'origin'

Cause: gh repo create tried to add a remote that already existed from a prior git remote add.

Fix: git remote remove origin → pushed directly with git push -u origin main

GraphQL: Name already exists on this account

Cause: The first gh repo create succeeded on GitHub before failing locally. Re-running tried to create a duplicate.

Fix: Skipped the create step — pushed directly since the repo already existed on GitHub

Nothing to commit (untracked files present)

Cause: All files were created at once. The first git add . had already included backend/index.js.

Fix: Reorganized remaining files into logical per-phase commit groups and pushed

Docker: permission denied (unix:///var/run/docker.sock)

Cause: The current Linux user was not part of the docker group.

Fix: sudo usermod -aG docker \$USER then newgrp docker to apply without logout

minikube: command not found

Cause: Minikube and kubectl were not installed on the Fedora system.

Fix: Installed both via official curl download from storage.googleapis.com and dl.k8s.io

Quiz not recognising correct answers

Cause: React stale state closure — the score value in handleNext captured a stale snapshot.

Fix: Simplified handleNext to call onComplete(score) directly, removing the redundant stale calculation

GitHub: <https://github.com/shashank-6832/quiz-app>

Shashank · Roll No. 231109 · CS515 Unix Programming · IIIT Tiruchirappalli · 29 March 2026